

LLMOrchestrator

LLMOrchestrator

One control plane for every headless CLI coding agent.

Summary

LLMOrchestrator is a standalone, reusable Go module for spawning, managing, and communicating with headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) over a hybrid pipe+file protocol, with per-agent circuit breakers, pluggable multi-provider selection, a decoupled i18n abstraction, and anti-bluff test guarantees.

Short description

A reusable Go module that gives one unified interface to spawn and drive multiple LLM-powered CLI agents via a hybrid pipe+file protocol. Thread-safe agent pooling with circuit breakers and selectable routing strategies, deliberately consumer-agnostic, with a pluggable i18n translator.

Long description

LLMOrchestrator is shared infrastructure for orchestrating headless CLI coding agents — the plumbing every multi-agent system quietly needs and usually rebuilds badly. Instead of each project re-implementing process spawning, message framing, and result parsing for tools like OpenCode, Claude Code, Gemini CLI, Junie, and Qwen Code, it provides one unified Agent interface, a thread-safe AgentPool, and a MultiProviderPool that marshals agents from several providers behind a single facade. Routing is pluggable via an AgentSelector — round-robin that skips providers failing to meet requirements, or preference-ordered with fallback — so how work is distributed is a policy you choose, not a hardcoded assumption. Each concrete agent is a thin adapter over

a shared BaseAdapter that owns the full process lifecycle: start with pipe setup, graceful SIGTERM-then-SIGKILL stop, restart, and liveness — the fiddly, error-prone part, solved once.

Communication is deliberately hybrid, matching the transport to the job. A pipe transport carries newline-delimited JSON with a per-request read deadline and a response-length cap for fast interactive messaging, while a file transport uses per-session inbox/outbox/shared directories for large or durable artifacts that shouldn't live in a pipe. Resilience isn't an afterthought — it's structural: a per-agent circuit breaker opens after three consecutive failures for a 60-second cooldown before a half-open probe, and a background health monitor pings agents so a downed agent can recover without waiting for incoming traffic to notice it. Pool acquisition blocks on a condition variable rather than burning CPU in a busy-wait, and the response parser is stateless and safe to call concurrently. The module is strictly decoupled — no consumer specifics are permitted to leak in — and every user-facing string passes through a pluggable i18n Translator, with a NoopTranslator that returns message IDs verbatim so a missing translation shows up loudly instead of hiding.

Why we built it

Every multi-agent system needs to launch and talk to CLI agents reliably. Re-solving spawning, framing, parsing, and failure-handling per project is wasteful and error-prone.

LLMOrchestrator centralizes it into one decoupled, reusable module whose specialised responsibility makes it reusable — and that reusability is destroyed the moment any consumer's specifics leak in.

Why it's a game-changer

It turns “run an army of heterogeneous CLI agents” from a bespoke per-project engineering slog into a single library import — pooling, circuit breaking, lifecycle management, and pluggable routing already solved and hardened. And because its anti-bluff tests exercise the real system end-to-end rather than settling for “it compiles,” you get an abstraction you can actually trust to work under concurrency and failure, not one that merely looks right in a diagram.

What's innovative

- **Hybrid pipe+file protocol** — interactive speed (JSON-lines over stdin/stdout, read deadlines, response caps) *and* durable

file-based exchange (inbox/outbox/shared) for large artifacts, so you never trade latency for durability or vice versa.

- **Multi-provider pool with pluggable selectors** — one facade over many CLI providers, with round-robin or preference-ordered routing chosen as policy rather than baked in.
- **Per-agent circuit breaker + background health monitor** — automatic degradation *and* recovery (3 failures → 60s open → half-open probe), so a flaky agent is isolated and then quietly brought back without manual intervention.
- **Non-busy-wait pooling** — Acquire blocks on `sync.Cond` until a matching, healthy agent frees up or the context is cancelled, so waiting costs no CPU.
- **Strict decoupling + anti-bluff i18n** — the `NoopTranslator` returns message IDs verbatim so a missing translation is impossible to miss instead of silently blank.
- **Security-by-default** — a binary-path allowlist means no shell interpolation and therefore no command-injection surface, backed by path-traversal protection, a 1 MiB response cap against runaway output, and API-key masking in logs.
- **Anti-bluff Challenge harness** — real disk/JSON/parser round-trips across five locales, with a paired mutation gate that must exit non-zero when the feature is broken — a test that proves it can actually fail.

Biggest technical challenges & how we solved them

- **Reliable agent process I/O.** Talking to a spawned CLI process is deceptively hard; solved with a hybrid pipe+file transport, a defined message/parser contract so both sides agree on the wire format, and a `BaseAdapter` that centralizes the whole process lifecycle including a graceful `SIGTERM` timeout that escalates to a `SIGKILL` fallback.
- **Concurrency without busy-waiting.** Solved with a mutex + condition-variable `AgentPool` where `Acquire` sleeps until a capability-matched agent actually frees up, paired with a stateless, side-effect-free parser that is safe to call from many goroutines at once.
- **Provider failure isolation.** Solved so one bad provider can't drag down the rest: per-agent circuit breakers contain the blast radius, and a health-monitor goroutine drives recovery even when no requests are arriving to trigger it.
- **Proving correctness, not just compilation.** Solved with a Challenge runner: dozens of invariants across `en/sr/ja/es/de` that exercise the real system, plus a paired mutation gate (`LLMORCH_MUTATE_RUNNER=1` must fail → wrapper exit 99) that deliberately breaks the feature to prove the gate itself isn't a bluff.
- **Localization without silent failure.** Solved with the verbatim-id `NoopTranslator` seam and per-consumer translator

injection, so a gap in translations is always visible rather than papered over.

Tech stack

- **Go (1.25)** — chosen for first-class concurrency and clean process control, which is exactly what orchestrating live agent processes demands; it implements the module, its agent adapters, transports, and parser.
- **Go stdlib only (+ testify, yaml.v3)** — a deliberate choice to keep the dependency surface minimal and pull in *no* LLM SDKs, so the module stays lightweight and embeddable inside any consumer without dragging vendor baggage along.
- **Pipe transport (JSON-lines over stdio)** — chosen for fast interactive messaging, hardened with read deadlines and response-length limits so a hung or runaway agent can't stall the caller.
- **File transport (inbox/outbox/shared)** — chosen for durable, large-artifact exchange per session, where a pipe would be the wrong tool.
- **sync.Mutex/sync.Cond** — chosen to implement blocking, fair agent-pool acquisition without busy-waiting.
- **Circuit breaker + HealthMonitor** — chosen together to deliver per-agent resilience *and* active recovery, not just failure detection.
- **pkg/i18n Translator** — chosen as the decoupled localization seam that keeps consumer-specific strings out of the core.
- **Challenge harness (challenges/runner) + Makefile (test -race, fuzz, cover)** — chosen for anti-bluff, evidence-backed verification, including race detection and parser fuzzing so correctness is demonstrated under adversarial conditions, not assumed.

Status & honesty notes

- **Status: beta.** A decoupled reusable module, consumed as a submodule by multiple Helix/vasic projects. **License: Apache-2.0**; the GitHub repo is public.
- Model metadata comes from LLMsVerifier bridged via HelixQA; this module does not import LLMsVerifier/VisionEngine/DocProcessor directly. Stacks referenced in the parent app's CLAUDE.md (Gin/PostgreSQL/etc.) describe `helix_code`, not this module.

Priority tier: Helix-primary (LLM-infrastructure cluster — decoupled reusable module). Ranks after HelixTrack.